Technical Summary: Secure Stripe Webhook Integration

Client	Zenith SaaS Startup Global
Project	Integrate Stripe Checkout and Payment API
Assignment	Secure Stripe payment webhook integration
Mode	Stripe test mode only
Deadline	13 November 2026
Stack	Node.js, Express.js, Stripe Node.js SDK, dotenv

---

1. Project Overview

This project is a backend service that receives webhook notifications from Stripe when a Checkout payment completes. It performs five jobs:

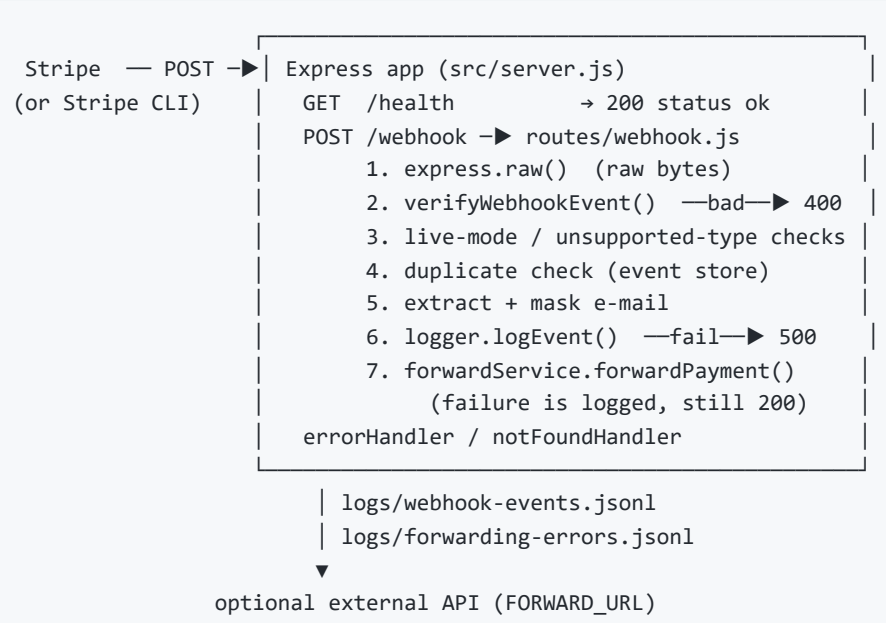
1. Accepts `POST /webhook` requests using the **raw** request body.
2. **Verifies** the `Stripe-Signature` header with `stripe.webhooks.constructEvent()`; invalid requests receive HTTP 400.
3. Handles `checkout.session.completed` events and extracts the payment details.
4. Writes the details as one JSON object per line (JSONL) to a local log file.
5. **Optionally forwards** the details to an external API configured only through environment variables.

There is no frontend. A `GET /health` endpoint reports that the service is running.

Scope note. The client has not supplied a forwarding endpoint, an API specification or a JSON schema. These were therefore **not invented**. Forwarding is disabled by default, and the default payload is the list of extracted fields from the brief. It can be replaced by editing one function once the client confirms a schema.

---

2. Architecture



Module responsibilities

File	Responsibility
`src/server.js`	Builds the app (`createApp`), validates startup settings, starts and stops the server
`src/routes/webhook.js`	The webhook request pipeline and HTTP status decisions
`src/services/stripeService.js`	Stripe client (test keys only), signature verification, field extraction, e-mail masking, duplicate-event store
`src/services/logger.js`	JSONL file logging plus console output
`src/services/forwardService.js`	Optional forwarding: configuration, timeout, status classification
`src/middleware/errorHandler.js`	Generic JSON errors and 404 handling
`tests/webhook.test.js`	40 automated tests

Settings are read from `process.env` when a function runs, not when a file is loaded. This keeps the code testable and avoids hardcoded values.

---

3. Dependencies

Package	Version installed	Purpose
`express`	5.3.0	HTTP server and routing
`stripe`	23.1.0	Official Stripe SDK (signature verification)
`dotenv`	18.0.10	Loads `.env` into environment variables

No other runtime dependencies. Tests use Node's built-in test runner (`node:test`), so there are no test dependencies. Runtime: Node.js 18.2 or newer (verified on Node v24.13.0 on Windows 11).

---

4. Endpoint Configuration

Method	Path	Description
`GET`	`/health`	Returns `200 {"status":"ok","service":...,"uptime_seconds":...,"timestamp":...}`
`POST`	`/webhook`	Stripe webhook receiver. Body size limit 1 MB. Requires `Content-Type: application/json` and a `Stripe-Signature` header
any other		`404 {"error":"Not found."}`

Environment variables are documented in `.env.example` and the README. Required: `STRIPE_SECRET_KEY` (test key) and `STRIPE_WEBHOOK_SECRET` (`whsec_...`). The server refuses to start if either is missing or malformed, or if a live key is supplied.

HTTP status policy for `POST /webhook`

Situation	Status	Reason
Valid, processed and logged	200	Stripe stops retrying
Unsupported event type	200	Received correctly; nothing to do. A non-2xx would cause days of pointless retries
Duplicate event ID	200	Already processed
Live-mode event	200	Received but not accepted (test mode only)
Invalid, missing or expired signature; non-JSON body	400	Caller cannot be trusted
Log file cannot be written; server misconfigured	500	Stripe will retry later, so a payment record is never silently lost
Forwarding failed	**200**	The event is verified and logged. A 500 would trigger Stripe retries and duplicates

---

5. Stripe Signature Verification

Stripe signs every webhook: the `Stripe-Signature` header contains a timestamp and an HMAC-SHA256 signature computed with the endpoint's signing secret over the **exact raw body**.

Implementation points:

- `express.raw({ type: 'application/json', limit: '1mb' })` is attached to the webhook route only, so the handler receives the original bytes as a `Buffer` . No global `express.json()` is registered before it, because parsing and re-serialising would change the bytes and break verification.

- `stripe.webhooks.constructEvent(rawBody, signature, secret)` verifies the signature and rejects timestamps older than the SDK tolerance (5 minutes), which blocks replayed messages.
- A verification error returns **HTTP 400** with a generic message. The log records only the reason code `invalid_signature`, never the signature, the body or the secret.
- A missing header or a non-JSON content type also returns 400.
- A missing server secret is a *server* fault and returns 500, with a generic body that does not reveal which setting is missing.

---

6. JSON Payload Processing

For `checkout.session.completed` events the service extracts only:

Field	Source
`event_id`	`event.id`
`event_type`	`event.type`
`checkout_session_id`	`data.object.id`
`payment_intent_id`	`data.object.payment_intent` (string, or `.id` if expanded)
`customer_email`	`customer_details.email`, falling back to `customer_email`; **masked by default**
`amount_total`	`data.object.amount_total` (smallest currency unit, e.g. 2000 = 20.00)
`currency`	`data.object.currency`
`payment_status`	`data.object.payment_status`

Missing values become `null` (for example when no e-mail was entered or the session has no payment intent). Everything else in the Stripe object (name, address, phone, billing details) is never read into the log record.

Unsupported types (for example `payment_intent.succeeded`) are acknowledged with 200, logged as `event_ignored` with only the event ID and type, and are never forwarded.

Duplicate events are detected through an event-ID store with a two-method interface (`has`, `add`). The default implementation is in memory (bounded to 10,000 IDs). An ID is stored only after its log line has been written successfully, so a failed attempt can be retried by Stripe.

---

7. Logging

Two JSON Lines files are written to `logs/` (or `LOG_DIR`), and each record is also printed to the console.

`webhook-events.jsonl` example:


```
{"timestamp":"2026-11-11T01:43:29.225Z","level":"info","record_type":"checkout_session_completed","event_
```

Record types: `checkout_session_completed`, `event_ignored`, `event_duplicate`, `webhook_rejected`, `forwarding_succeeded`, `server_error`.

`forwarding-errors.jsonl` holds **only** failures (`record_type: forwarding_failed`) with `event_id`, `error_type`, `status_code`, `message`, `duration_ms` and `target_host`.

Data minimisation: the logs never contain the raw body, the signature header, API keys, the webhook secret, the auth header value, the full forwarding URL (only the host), or customer name/address. E-mail is masked (`j***@example.com`) unless `LOG_FULL_EMAIL=true`. Automated tests assert each of these.

---

8. Optional API Forwarding

Controlled entirely by environment variables: `FORWARD_ENABLED`, `FORWARD_URL`, `FORWARD_METHOD` (POST/PUT/PATCH), `FORWARD_TIMEOUT_MS` (default 5000; clamped to 100–30000), optional `FORWARD_AUTH_HEADER_NAME` / `_VALUE`, and `FORWARD_FULL_EMAIL`.

- Uses `fetch` with an `AbortController` timeout. Redirects are not followed.
- The function **never throws**; it returns a result object, so forwarding can never break the webhook response.
- Outcome classes: `success` (2xx), `redirect` (3xx), `rate_limited` (429), `client_error` (other 4xx), `server_error` (5xx), `timeout`, `network` (DNS/connection/TLS), `config` (bad settings).
- Failures are written to `forwarding-errors.jsonl` and the webhook still answers 200.
- The payload is built in one function, `buildForwardPayload()`. It currently sends the extracted fields. **Replace it once the client supplies a schema.**

---

9. Security Practices

Practice	Implementation
Authenticity	Signature verified on the raw body for every request; timestamp tolerance blocks replay
No hardcoded secrets	All keys and secrets come from environment variables; `.env` is git-ignored; `.env.example` has placeholders only
Test mode only	Startup rejects non-test keys; live-mode events are ignored
Fail closed	Any verification failure returns 400 and does no processing
Safe errors	Callers see generic messages; details only in the server log; no stack traces
Privacy	E-mail masked by default; unused personal data is never read; full e-mail is an explicit opt-in (separately for log and forwarding)
Secret-safe logging	Auth header values and URL query strings are never logged
Redirect safety	Forwarding does not follow redirects, so credentials cannot be sent to another host
Reduced fingerprinting	`X-Powered-By` header disabled
Input limits	1 MB body limit on the webhook route
Idempotency	Duplicate event IDs are skipped

---

10. Testing Procedure

1. **Automated:** `npm test` (40 tests; signed with the Stripe SDK test helper; fake local external API; isolated log folder per test).
2. **Process-level manual test:** start `npm start`, call the endpoints with `curl`, generate a signed request, and inspect the log files.
3. **Postman:** import the collection and run it (health 200; fake and missing signature 400). Postman cannot create genuine signatures; this is expected.
4. **Stripe CLI (genuine verification), to be run on the developer's machine:** `stripe login` → `stripe listen --forward-to localhost:3000/webhook` (copy the `whsec_` secret to `.env`) → `npm start` → `stripe trigger checkout.session.completed` → check the terminals and `logs/webhook-events.jsonl`.

---

11. Results and Limitations

Results (executed in the development environment, Node v24.13.0 on Windows)

Check	Result
Automated suite (`npm test`)	**40 of 40 passed**, 0 failed
Mutation check: signature verification deliberately disabled	4 tests failed as expected; after restoring, 40 of 40 passed (proves the tests detect broken security)
Invalid / fake / tampered / expired signature	HTTP 400, nothing logged as a payment, nothing forwarded
Missing signature header; non-JSON content type	HTTP 400
Valid signed `checkout.session.completed`	HTTP 200; correct fields written to JSONL; e-mail masked; no name/address in logs
Unsupported event type	HTTP 200, logged as `event_ignored` , not forwarded
Duplicate event ID	HTTP 200 `duplicate:true` ; logged once; forwarded once
Forwarding failures (500, 400, 429, redirect, timeout, unreachable host, bad config)	Webhook still HTTP 200; failure recorded in `forwarding-errors.jsonl` with the correct `error_type`
Log write failure	HTTP 500; the same event succeeded when retried
Startup safety	Refuses to start with no settings or with a live key (exit code 1)
Real server process (`npm start`) with `curl`	Health 200; unsigned 400; signed 200 with forwarding to a fake API (auth header delivered); duplicate skipped; graceful shutdown on SIGTERM
Postman collection (run with Newman 6.2.3 against the live server)	4 requests, 6 assertions, 0 failures

Limitations (stated honestly)

- **Stripe CLI test was not run by the assistant.** It needs the developer's Stripe login and a connection to Stripe. Signed requests in the tests use the SDK's official test-signature helper and a fake secret, which exercises the real verification code but does not prove that Stripe's servers reached the machine. The developer must run the CLI steps in section 10 and record the result.
- **Postman Desktop was not used.** The collection was validated as JSON and executed with Newman (Postman's official command-line runner), which parses the same format.
- **Tested on Node 22 only.** Node 18.2+ is declared as the minimum but was not run.
- **Duplicate detection is in memory:** lost on restart, not shared across several instances. Upgrade path: implement `has(id)` / `add(id)` against Redis or a database and call `setEventStore()` .
- **No automatic forwarding retries.** A failed forward is logged but not re-sent. A queue with retries is a recommended extension if delivery must be guaranteed.
- **Forwarding is synchronous** (the response to Stripe waits for the forward, at most the configured timeout, default 5 s). Moving it to a background queue is a sensible future improvement.
- **Out of scope for this assignment:** HTTPS termination (provided by the host or a reverse proxy), rate limiting, log rotation, monitoring and alerting.

- **Client information still missing:** forwarding URL, method, JSON schema, authentication method, whether forwarding is required, and whether full e-mail may be stored or sent.